

LAPORAN PRAKTIKUM STRUKTUR DATA

Queue

DISUSUN OLEH:

Rayya Syaquinah Putri Hasibuan

2511532007

DOSEN PENGAMPU:

Dr. Wahyudi, S.T. M.T

ASISTEN PRATIKUM:

M. Galid



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban pelaksanaan praktikum struktur data dengan judul Stack. Laporan ini bertujuan untuk mendokumentasikan prosedur, hasil, serta pembahasan dari kegiatan praktikum sekaligus sebagai sarana pembelajaran bagi penulis untuk lebih memahami materi terkait.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu. Saran dan kritik yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu sehingga laporan ini dapat terselesaikan.

Padang, 27 April 2026

Rayya Syaquinah Putri Hasibuan

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	3
2.1 Deskripsi Materi Praktikum	3
2.2 Queue dalam Java Script.....	4
a. QueueArray_2511532007	4
b. QueueArrayDriver_2511532007.....	6
c. QueueLinkedList_2511532007	7
d. ReverseData_2511532007	9
e. IterasiQueue_2511532007	10
BAB III KESIMPULAN DAN SARAN	11
3.1 Kesimpulan	11
3.2 Saran.....	12
DAFTAR PUSTAKA.....	12
TUGAS PRAKTIKUM PEKAN 4	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan konsep penting dalam ilmu komputer yang digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses secara efisien. Salah satu struktur data dasar yang sering digunakan adalah queue (antrian), yaitu struktur data linear yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari maupun dalam sistem komputer, seperti antrian printer, penjadwalan proses, dan pengolahan data berurutan.

Dalam pembelajaran struktur data, queue menjadi materi penting karena memiliki perbedaan konsep dengan struktur data lain seperti stack. Selain itu, implementasi queue dapat dilakukan dengan berbagai cara, seperti menggunakan array maupun linked list. Oleh karena itu, diperlukan praktikum agar mahasiswa tidak hanya memahami konsep secara teori, tetapi juga mampu mengimplementasikan dan menggunakannya dalam pemrograman secara langsung.

1.2 Tujuan Praktikum

Tujuan dari praktikum struktur data queue adalah untuk membantu mahasiswa memahami konsep dasar queue yang bekerja dengan prinsip FIFO (First In First Out), serta mampu mengimplementasikannya dalam bahasa pemrograman seperti Java. Melalui praktikum ini, mahasiswa diharapkan dapat membuat dan menggunakan operasi dasar queue seperti enqueue, dequeue, front, dan rear, baik menggunakan array maupun linked list. Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan mahasiswa

dalam menerapkan konsep queue untuk menyelesaikan permasalahan sederhana secara logis dan sistematis

1.3 Manfaat Praktikum

Manfaat dari praktikum struktur data queue adalah meningkatkan pemahaman mahasiswa terhadap pengelolaan data secara terstruktur dan berurutan. Praktikum ini juga membantu mahasiswa menghubungkan teori dengan implementasi nyata dalam pemrograman, sehingga kemampuan teknis menjadi lebih terasah. Selain itu, mahasiswa dapat memahami bagaimana queue digunakan dalam berbagai sistem nyata seperti antrian layanan, penjadwalan proses, dan pengolahan data, serta melatih kemampuan berpikir logis dan pemecahan masalah dalam pengembangan program.

BAB II

PEMBAHASAN

2.1 Deskripsi Materi Praktikum

Struktur data queue (antrian) merupakan salah satu struktur data linear yang digunakan untuk menyimpan dan mengelola data secara berurutan dengan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Dalam queue, terdapat dua bagian utama yaitu front (depan) sebagai tempat pengambilan data, dan rear (belakang) sebagai tempat penambahan data. Konsep ini mirip dengan antrian di kehidupan sehari-hari, seperti antrian di kasir atau loket layanan.

Dalam struktur data queue terdapat beberapa operasi dasar, yaitu enqueue (menambahkan data ke bagian belakang), dequeue (menghapus data dari bagian depan), serta operasi tambahan seperti peek/front untuk melihat data terdepan tanpa menghapusnya, isEmpty untuk mengecek apakah antrian kosong, dan isFull untuk mengecek apakah antrian penuh. Operasi-operasi ini menjadi dasar dalam implementasi queue pada program.

Queue dapat diimplementasikan dengan beberapa cara, di antaranya menggunakan array dan linked list. Pada implementasi array, queue biasanya memiliki kapasitas tetap sehingga perlu pengelolaan indeks seperti front dan rear, bahkan dapat dikembangkan menjadi circular queue agar penggunaan memori lebih efisien. Sedangkan pada linked list, queue bersifat lebih fleksibel karena ukuran dapat bertambah sesuai kebutuhan. Dalam bahasa pemrograman seperti Java, queue juga sudah tersedia dalam bentuk bawaan melalui interface Queue yang dapat diimplementasikan menggunakan LinkedList.

Dalam penerapannya, queue banyak digunakan dalam berbagai sistem komputer, seperti antrian printer, penjadwalan proses (CPU

scheduling), buffering data, dan sistem pelayanan. Oleh karena itu, pemahaman tentang queue sangat penting karena menjadi dasar dalam pengembangan program dan sistem yang membutuhkan pengolahan data secara teratur dan berurutan.

2.2 Queue dalam Java Script

a. QueueArray_2511532007



```

1 package pekan4_2511532007;
2
3 public class QueueArray_2511532007 {
4     int front_2007, rear_2007, size_2007;
5     int capacity_2007;
6     int array_2007[];
7
8     public QueueArray_2511532007(int capacity_2007) {
9         this.capacity_2007 = capacity_2007;
10        front_2007 = this.size_2007 = 0;
11        array_2007 = new int[this.capacity_2007];
12    }
13    boolean isFull(QueueArray_2511532007 queue) {
14        return (queue.size_2007 == queue.capacity_2007);
15    }
16    boolean isEmpty(QueueArray_2511532007 queue) {
17        return (queue.size_2007 == 0);
18    }
19    void enqueue_2007(int item) {
20        if (isFull(this))
21            return;
22        this.rear_2007 = (this.rear_2007 + 1) % this.capacity_2007;
23        this.array_2007[this.rear_2007] = item;
24        this.size_2007 = this.size_2007 + 1;
25        System.out.println(item + " enqueued to queue");
26    }
27    int dequeue_2007 () {
28        if (isEmpty(this))
29            return Integer.MIN_VALUE;
30        int item = this.array_2007[this.front_2007];
31        this.front_2007 = (this.front_2007 + 1) % this.capacity_2007;
32        this.size_2007 = this.size_2007 - 1;
33        return item;
34    }
35
36    int front_2007() {
37        if (isEmpty(this))
38            return Integer.MIN_VALUE;
39        return this.array_2007[this.front_2007];
40    }
41    int rear_2007() {
42        if (isEmpty(this))
43            return Integer.MIN_VALUE;
44        return this.array_2007[this.rear_2007];
45    }
46    //mencetak elemen antrian
47    void display_2007() {
48        int i_2007;
49        if (front_2007 == rear_2007) {
50            System.out.println("Antrian Kosong\n");
51            return;
52        }
53        //mengikuti dari belakang dan cetak
54        for (i_2007 = front_2007; i_2007 < rear_2007; i_2007++) {
55            System.out.printf("%d <- ", array_2007[i_2007]);
56        }
57        return;
58    }
59 }
60
61

```

Langkah – Langkah Beserta Penjelasan Kode Program:

1. Deklarasi awal program, package pekan4_2511532007; digunakan untuk mengelompokkan file Java.
public class QueueArray_2511532007 adalah kelas utama yang berisi implementasi struktur data queue menggunakan array.
2. Deklarasi variabel dan atribut, front menunjukkan elemen paling depan(yang akan keluar duluan), rear menunjukkan elemen paling belakang(tempat masuk data baru), size untuk

jumlah elemen dalam queue, capacity untuk kapasitas maksimum queue, array sebagai tempat menyimpan data. Setiap variabel diberikan ujung angka 2007(NIM akhir) dari programmer.

3. Konstruktor/ inisialisasi awal 'public QueueArray_2511532007(int capacity_2007)' untuk mengatur kapasitas queue. Front = 0 dan size = 0 menandakan queue kosong dan membuat array sesuai kapasitas.
4. 'boolean isFull(QueueArray_2511532007 queue)' berfungsi untuk mengecek apakah queue penuh, maka kondisi = size == capacity dengan arti jumlah elemen dalam queue sama dengan kapasitas maksimum yang bisa ditampung, sehingga jika antrian dalam keadaan penuh, tidak dapat menerima data baru lagi.
5. 'boolean isEmpty(QueueArray_2511532007 queue)' untuk mengecek apakah queue kosong, maka kondisi size == 0 dengan arti tidak ada elemen sama sekali dalam queue.
6. Method enqueue(menambah data) 'void enqueue_2007(int item)' untuk mengecek apakah queue penuh, kalau penuh, data tidak bisa dimasukkan. Jika tidak, proses penambahan bisa dilakukan, maka dibuat kode 'rear = (rear + 1) % capacity' agar array bisa dipakai berulang seperti lingkaran(*circular queue*). Setelah posisi rear ditentukan, barulah data dimasukkan ke array[rear] dan size ditambah karena jumlah data bertambah.
7. Method dequeue(menghapus data) 'int dequeue_2007()' mengecek apakah queue kosong. Jika tidak, data akan diambil dari posisi front 'int item = this.array_2007[this.front_2007];' dengan arti mengambil elemen yang paling lama masuk. Setelah data depan diambil, posisi depan pindah ke elemen berikutnya dengan kode

program 'this.front_2007 = (this.front_2007 + 1) % this.capacity_2007;'. Dengan kode 'this.size_2007 = this.size_2007 - 1;' artinya jumlah data berkurang satu.

8. Method front 'int front_2007()' untuk mengambil nilai paling depan tanpa menghapusnya.
9. Method rear 'int rear_2007()' untuk mengambil nilai paling belakang.
10. Method 'void display_2007()' untuk menampilkan isi queue, isi antrian dari depan ke belakang. Jika kosong, tampilkan "Antrian Kosong". Dan akan mengulang (loop) dari front ke rear.

b. QueueArrayDriver_2511532007

```

1 package pekani_2511532007;
2
3 public class QueueArrayDriver_2511532007 {
4
5     public static void main(String[] args) {
6         QueueArray_2511532007 queue = new QueueArray_2511532007(1000);
7         queue.enqueue_2007(10);
8         queue.enqueue_2007(20);
9         queue.enqueue_2007(30);
10        queue.enqueue_2007(40);
11        System.out.println("Item di depan " + queue.front_2007());
12        System.out.println("Item paling belakang " + queue.rear_2007());
13        System.out.println("Tampilkan Queue ");
14        queue.display_2007();
15        System.out.println();
16        System.out.println(queue.dequeue_2007() + "dihapus dari queue");
17        System.out.println("Item di depan " + queue.front_2007());
18        System.out.println("Item paling belakang " + queue.rear_2007());
19        System.out.println("Tampilkan Queue setelah satu data dihapus");
20        queue.display_2007();
21    }
22 }

```

Langkah – Langkah beserta Penjelasan Kode Program:

1. Deklarasi awal (package dan kelas)
2. 'QueueArray_2511532007 queue = new QueueArray_2511532007(1000);' objek queue dibuat dengan kapasitas maksimum 1000.
3. Dimasukkan empat data ke queue secara berurutan, karena queue menggunakan prinsip FIFO.
'queue.enqueue_2007(10);', 'queue.enqueue_2007(20);', 'queue.enqueue_2007(30);', 'queue.enqueue_2007(40);'.
4. 'System.out.println("Item di depan " + queue.front_2007());' akan menampilkan 10(data yang pertama masuk) dan 'System.out.println("Item paling belakang " +

- queue.rear_2007());’ menampilkan 40(data terakhir yang masuk).
5. ’ queue.display_2007());’ untuk menampilkan seluruh isi queue.
 6. ‘System.out.println(queue.dequeue_2007() + "dihapus dari queue");’ operasi ini menghapus satu data dari queue karena FIFO yang dihapus Adalah 10, bukan 40. Maka setelah kondisi ini output queue akan berubah.
 7. Setelah penghapusan, ‘System.out.println("Item di depan " + queue.front_2007());’, angka depan(front) menjadi 20, dan ‘System.out.println("Item paling belakang " + queue.rear_2007());’ angka belakang(rear) tetap 40
 8. ‘queue.display_2007());’ menampilkan isi queue setelah satu data dihapus.
 9. Maka outputnya:

```

251... x 2 Rev Probl... Deca... Cons... X Prog... GR SF... Cove... Prop...
251532007 {
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
Tampilkan Queue
10 <-- 20 <-- 30 <-- 40
dihapus dari queue
Item di depan 20
Item paling belakang 40
Tampilkan Queue setelah satu data dihapus
10 <-- 20 <-- 30 <--

```

Program driver ini menunjukkan alur kerja queue yang dimana adanya data yang masuk dari belakang(enqueue) dan adanya data yang keluar dari depan(dequeue).

c. QueueLinkedList_2511532007

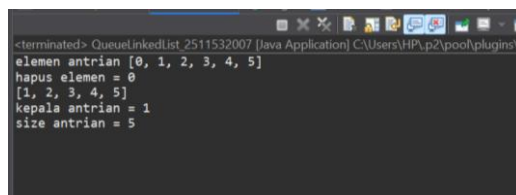
```

1 package pekand_2511532007;
2 import java.util.Queue;
3
4
5 public class QueueLinkedList_2511532007 {
6     public static void main(String[] args) {
7         Queue<Integer> q_2007 = new LinkedList<>();
8         // tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
9         for (int i_2007 = 0; i_2007 < 6; i_2007++)
10             q_2007.add(i_2007);
11         // menampilkan isi antrian
12         System.out.println("elemen antrian " + q_2007);
13         // untuk menghapus kepala antrian
14         int hapus = q_2007.remove();
15         System.out.println("hapus elemen = " + hapus);
16         System.out.println(q_2007);
17         // untuk melihat antrian terdepan
18         int depan = q_2007.peek();
19         System.out.println("kepala antrian = " + depan);
20
21         int banyak = q_2007.size();
22         System.out.println("size antrian = " + banyak);
23     }
24 }

```

Langkah – Langkah beserta Penjelasan Kode Program:

1. Deklarasi package, kelas dan import java.util.Queue; untuk memanggil struktur data queue bawaan java.
2. Deklarasi dan pembuatan objek queue ' Queue<Integer> q_2007 = new LinkedList<>();' dengan tipe data Integer. Artinya, queue hanya bisa menyimpan bilangan bulat.
3. Menambahkan elemen dalam queue ' for (int i_2007 = 0; i_2007 < 6; i_2007++), q_2007.add(i_2007);' yang berarti perulangan ini menambahkan elemen dari 0 sampai 5 kedalam queue. Dan method add berfungsi untuk menambah elemen ke bagian belakang(rear).
4. 'System.out.println("elemen antrian " + q_2007);' menampilkan isi queue dalam bentuk list [0, 1, 2, 3, 4, 5]
5. Menghapus elemen (dequeue). ' int hapus = q_2007.remove();' method remove() akan mengambil elemen paling depan dan sekaligus menghapusnya dari queue. Jadi yang dihapus adalah 0, bukan 5.
6. 'System.out.println(q_2007);' menampilkan queue setelah penghapusan.
7. 'int depan = q_2007.peek();' melihat elemen terdepan tanpa menghapus method peek() hanya melihat elemen paling depan tanpa menghapusnya. Jadi hasilnya adalah 1.
8. Menghitung jumlah elemen dalam queue 'int banyak = q_2007.size();' Method size() mengembalikan jumlah elemen dalam queue. Setelah satu elemen dihapus, jumlahnya menjadi 5.
9. Maka output yang dihasilkan:



```
<terminated> QueueLinkedList 2511532007 [Java Application] C:\Users\HP\p2\poo\plugins\d
elemen antrian [0, 1, 2, 3, 4, 5]
hapus elemen = 0
[1, 2, 3, 4, 5]
kepala antrian = 1
size antrian = 5
```

d. ReverseData_2511532007

```

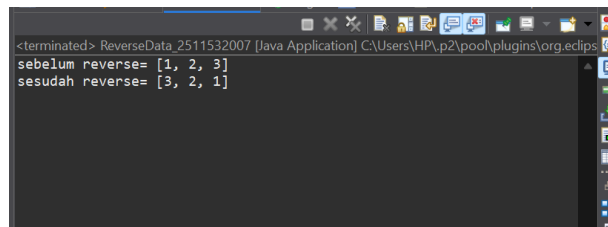
1 package pekad_2511532007;
2 import java.util.LinkedList;
3 import java.util.Queue;
4 import java.util.Stack;
5
6 public class ReverseData_2511532007 {
7     public static void main(String[] args) {
8         Queue<Integer> q_2007 = new LinkedList<Integer>();
9         q_2007.add(1);
10        q_2007.add(2);
11        q_2007.add(3); // [1, 2, 3]
12        System.out.println("sebelum reverse= " + q_2007);
13        Stack<Integer> s_2007 = new Stack<Integer>();
14        while (!q_2007.isEmpty()) { // Q -> S
15            s_2007.push(q_2007.remove());
16        }
17        while (!s_2007.isEmpty()) { // S -> Q
18            q_2007.add(s_2007.pop());
19        }
20        System.out.println("sesudah reverse= " + q_2007); // [3, 2, 1]
21    }
22 }

```

Langkah – langkah beserta penjelasan Kode Program:

1. Deklarasi package, kelas dan import java.util.Queue;, java.util.LinkedList;, dan java.util.Stack; untuk menggunakan struktur data queue dan stack bawaan Java.
2. Deklarasi dan pembuatan objek queue Queue<Integer> q_2007 = new LinkedList<Integer>(); dengan tipe data Integer, lalu menambahkan elemen q_2007.add(1);, q_2007.add(2);, q_2007.add(3); sehingga isi queue menjadi [1, 2, 3].
3. System.out.println("sebelum reverse= " + q_2007); menampilkan isi queue sebelum dibalik, yaitu [1, 2, 3].
4. Deklarasi stack Stack<Integer> s_2007 = new Stack<Integer>(); yang digunakan sebagai struktur bantu untuk membalik data.
5. Perulangan while (!q_2007.isEmpty()) { s_2007.push(q_2007.remove()); } memindahkan seluruh elemen dari queue ke stack. Data diambil dari depan queue lalu dimasukkan ke stack, sehingga urutannya terbalik karena sifat stack (LIFO).
6. Perulangan while (!s_2007.isEmpty()) { q_2007.add(s_2007.pop()); } memindahkan kembali data dari stack ke queue. Karena stack mengeluarkan data terakhir terlebih dahulu, maka urutan queue menjadi terbalik.

7. `System.out.println("sesudah reverse= " + q_2007);`
menampilkan hasil akhir queue yaitu [3, 2, 1].
8. Maka outputnya:



```
<terminated> ReverseData_2511532007 [Java Application] C:\Users\HP\p2\pool\plugins\org.edips
sebelum reverse= [1, 2, 3]
sesudah reverse= [3, 2, 1]
```

Program ini membalik urutan data dalam queue dengan memanfaatkan stack sebagai perantara. Queue memiliki sifat FIFO, sedangkan stack memiliki sifat LIFO. Ketika data dipindahkan ke stack lalu dikembalikan lagi ke queue, urutan data berubah menjadi kebalikan dari semula. Jadi, proses reverse tidak terjadi secara langsung, tetapi melalui perbedaan cara kerja dua struktur data tersebut.

e. IterasiQueue_2511532007



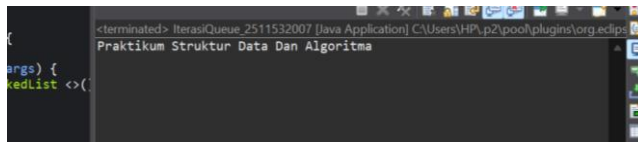
```
1 package pekand_2511532007;
20 import java.util.Iterator;
5 public class IterasiQueue_2511532007 {
6
7     public static void main(String[] args) {
8         Queue<String> q_2007 = new LinkedList<>();
9         q_2007.add("Praktikum");
10        q_2007.add("Struktur");
11        q_2007.add("Data");
12        q_2007.add("Dan");
13        q_2007.add("Algoritma");
14        Iterator<String> iterator = q_2007.iterator();
15        while (iterator.hasNext()) {
16            System.out.print(iterator.next() + " ");
17        }
18    }
19 }
20 }
```

Langkah – Langkah beserta Penjelasan Kode Program:

1. Deklarasi package, kelas dan import `java.util.Queue`;;
`java.util.LinkedList`;; dan `java.util.Iterator`;; untuk
menggunakan queue dan iterator bawaan Java.
2. Deklarasi dan pembuatan objek queue `Queue<String>`
`q_2007 = new LinkedList<>()`; dengan tipe data String.
3. Menambahkan elemen ke dalam queue
`q_2007.add("Praktikum")`;; `q_2007.add("Struktur")`;;
`q_2007.add("Data")`;; `q_2007.add("Dan")`;;

`q_2007.add("Algoritma");` sehingga isi queue menjadi [Praktikum, Struktur, Data, Dan, Algoritma].

4. Membuat iterator `Iterator<String> iterator = q_2007.iterator();` yang digunakan untuk menelusuri elemen dalam queue tanpa menghapusnya.
5. Perulangan `while (iterator.hasNext())`
`{ System.out.print(iterator.next() + " "); }` digunakan untuk mengambil dan menampilkan setiap elemen queue satu per satu dari depan ke belakang. Method `hasNext()` mengecek apakah masih ada elemen, sedangkan `next()` mengambil elemen berikutnya.
6. Output yang dihasilkan:



Program ini digunakan untuk menelusuri isi queue menggunakan iterator tanpa mengubah atau menghapus data di dalamnya. Berbeda dengan operasi `dequeue` yang menghapus elemen, iterasi hanya membaca data. Dengan demikian, isi queue tetap utuh setelah proses iterasi selesai.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Praktikum ini menunjukkan bahwa struktur data queue bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi yang pertama keluar. Melalui implementasi menggunakan array, linked list, serta pemanfaatan library Java, dapat dipahami bahwa setiap metode memiliki karakteristik dan kelebihan masing-masing. Selain itu, penggunaan operasi seperti enqueue, dequeue, dan iterasi membantu memperjelas bagaimana queue digunakan dalam pengolahan data secara terstruktur. Praktikum ini juga menegaskan bahwa pemahaman konsep sangat penting agar implementasi program tidak hanya berjalan, tetapi juga benar secara logika.

3.2 Saran

Mahasiswa disarankan untuk membandingkan beberapa metode implementasi agar lebih memahami kelebihan dan kekurangannya. Penting juga untuk membiasakan penulisan kode yang rapi dan pengujian program secara menyeluruh agar hasil yang diperoleh lebih optimal.

DAFTAR PUSTAKA

- [1] DosenIT, “Struktur Data Queue: Pengertian, Jenis dan Contoh,” 2023. [Online]. Available: <https://dosenit.com/ilmu-komputer/struktur-data-queue>. [Accessed: Apr. 27, 2026].
- [2] Staditek, “Mengenal Struktur Data Queue: Jenis, Fungsi, Keuntungannya,” 2024. [Online]. Available: <https://www.staditek.id/insight/arti-struktur-data-queue/>. [Accessed: Apr. 27, 2026].
- [3] Course-Net, “Struktur Data Queue: Pengertian, Jenis & Contoh,” 2024. [Online]. Available: <https://course-net.com/blog/struktur-data-queue-pengertian-jenis-contoh-struktur-data-queue-dalam-mengolah-data/>. [Accessed: Apr. 27, 2026].
- [4] Binus University, “Queue (Struktur Data),” 2018. [Online]. Available: <https://socs.binus.ac.id/2018/12/21/queue/>. [Accessed: Apr. 29, 2026].
- [5] Kompas.com, “Pengertian Stack dan Queue serta Contoh Penerapannya dalam Pemrograman,” 2024. [Online]. Available: <https://tekno.kompas.com/read/2024/09/08/03350097/pengertian-stack-dan-queue-serta-contoh-penerapannya-dalam-pemograman>. [Accessed: Apr. 29, 2026].
- [6] Codepolitan, “Apa itu Struktur Data Queue: Pengertian, Jenis dan Kegunaannya,” 2025. [Online]. Available: <https://www.codepolitan.com/blog/apa-itu-struktur-data-queue-pengertian-jenis-dan-kegunaannya/>. [Accessed: Apr. 29, 2026].

TUGAS PRAKTIKUM PEKAN 4

1. Kode Program

- a. Kode program objek:

```

public class Queue_2511532007 {
    int front_2007;
    int rear_2007;
    int max_2007;
    String queue_2007[];

    public Queue_2511532007(int max_2007) {
        this.max_2007 = max_2007;
        queue_2007 = new String[max_2007];
        front_2007 = -1;
        rear_2007 = -1;
    }

    boolean isEmpty_2007() {
        return rear_2007 == -1;
    }

    boolean isFull_2007() {
        return rear_2007 == max_2007 - 1;
    }

    void enqueue_2007(String data_2007) {
        if (isFull_2007()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_2007()) {
                front_2007 = 0;
            }
            rear_2007++;
            queue_2007[rear_2007] = data_2007;
        }
    }
}

```

```

        System.out.println("Data berhasil ditambahkan ke
antrian");
    }
}

void dequeue_2007() {
    if (isEmpty_2007()) {
        System.out.println("Antrian kosong!");
    } else {
        String keluar_2007 = queue_2007[front_2007];
        System.out.println(keluar_2007 + " telah dilayani");

        if (front_2007 == rear_2007) {
            front_2007 = -1;
            rear_2007 = -1;
        } else {
            front_2007++;
        }
    }
}

void display_2007() {
    if (isEmpty_2007()) {
        System.out.println("Antrian kosong");
    } else {
        System.out.println("Isi antrian:");
        for (int i_2007 = front_2007; i_2007 <= rear_2007;
i_2007++) {
            System.out.println((i_2007 - front_2007 + 1) + ". " +
queue_2007[i_2007]);
        }
    }
}

```

```

    }

    void reverse_2007() {
        if (isEmpty_2007()) {
            System.out.println("Antrian kosong");
        } else {
            int start_2007 = front_2007;
            int end_2007 = rear_2007;

            while (start_2007 < end_2007) {
                String temp_2007 = queue_2007[start_2007];
                queue_2007[start_2007] = queue_2007[end_2007];
                queue_2007[end_2007] = temp_2007;

                start_2007++;
                end_2007--;
            }
            System.out.println("Antrian berhasil dibalik");
        }
    }
}

```

b. Kode Program Driver

```

import java.util.Scanner;

public class AntrianLoket_2511532007 {
    public static void main(String[] args) {
        Scanner input_2007 = new Scanner(System.in);

        System.out.print("Masukkan kapasitas antrian: ");
        int kapasitas_2007 = input_2007.nextInt();
        input_2007.nextLine();
    }
}

```

```

Queue_2511532007 antrian_2007 = new
Queue_2511532007(kapasitas_2007);

int pilihan_2007;

do {
    System.out.println("\n=== PROGRAM ANTRIAN
LOKET ===");
    System.out.println("1. Tambah Antrian");
    System.out.println("2. Hapus Antrian");
    System.out.println("3. Tampilkan Antrian");
    System.out.println("4. Reverse");
    System.out.println("5. Keluar");
    System.out.print("Pilih menu: ");
    pilihan_2007 = input_2007.nextInt();
    input_2007.nextLine();

    switch (pilihan_2007) {
        case 1:
            System.out.print("Masukkan nama pelanggan: ");
            String nama_2007 = input_2007.nextLine();
            antrian_2007.enqueue_2007(nama_2007);
            break;
        case 2:
            antrian_2007.dequeue_2007();
            break;
        case 3:
            antrian_2007.display_2007();
            break;
        case 4:
            antrian_2007.reverse_2007();
            break;
    }
}

```

case 5:

```
System.out.println("Program selesai");
```

```
break;
```

default:

```
System.out.println("Pilihan tidak valid");
```

```
}
```

```
} while (pilihan_2007 != 5);
```

```
}
```

```
}
```

2. Output

```

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: andi
Data berhasil ditambahkan ke antrian

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: budi
Data berhasil ditambahkan ke antrian

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. andi
2. budi
  
```

```

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: andi
Data berhasil ditambahkan ke antrian

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. andi
  
```

```

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: andi
Data berhasil ditambahkan ke antrian

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. andi

===== PROGRAM ANTRIAN LOKET =====
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
  
```

3. Kesimpulan program:

Program yang telah dibuat merupakan implementasi struktur data Queue (antrian) menggunakan array dalam bahasa Java, yang disesuaikan langsung dengan kode program yang telah dikembangkan. Program ini bekerja dengan

prinsip FIFO (First In First Out), di mana data pelanggan yang pertama masuk akan menjadi yang pertama dilayani. Dalam kode, antrian direpresentasikan menggunakan array `queue_2007[]` dengan dua variabel utama yaitu `front_2007` sebagai penunjuk elemen depan dan `rear_2007` sebagai penunjuk elemen belakang, serta `max_2007` sebagai kapasitas maksimal antrian. Program menyediakan beberapa method seperti `enqueue_2007()` untuk menambahkan data, `dequeue_2007()` untuk menghapus data, `display_2007()` untuk menampilkan isi antrian, `isEmpty_2007()` dan `isFull_2007()` untuk mengecek kondisi antrian, serta `reverse_2007()` untuk membalik urutan data dalam antrian.

Selain itu, program juga telah dipisahkan menjadi dua class, yaitu class `Queue_2511532007` sebagai class objek yang menangani seluruh proses logika antrian, dan class `AntrianLoket_2511532007` sebagai driver yang mengatur jalannya program melalui menu interaktif. Dengan demikian, program ini tidak hanya memenuhi seluruh kebutuhan fungsional tugas, tetapi juga telah menerapkan konsep pemrograman berorientasi objek secara baik melalui pemisahan antara logika dan antarmuka pengguna.